Actividad 4

Decodificador de 7 segmentos

Índice

1. Introducción y objetivos 2
2. Creación del proyecto 3
3. Creación del código VHDL del decodificador 3
4. Simulación del decodificador 7
5. Asignación de pines de la FPGA 10
6. Configuración del decodificador en la FPGA 12
7. Niveles de abstracción en VHDL 12
8. Descripción funcional del decodificador de 7 segmentos 14
9. Descripción estructural del decodificador de 7 segmentos 16
Referencias 20

Actividad 4

Decodificador de 7 segmentos

1.- Introducción y objetivos

En esta práctica se va a proceder a implementar en la FPGA de la tarjeta de desarrollo un decodificador de 7 segmentos, un circuito ampliamente usado para poder visualizar números.

El decodificador de 7 segmentos es un circuito digital que convierte números BCD (*binary-coded decimal*, decimal codificado en binario, valores del 0 al 9) para adaptarlos a un display de 7 segmentos como el de la Figura 1.

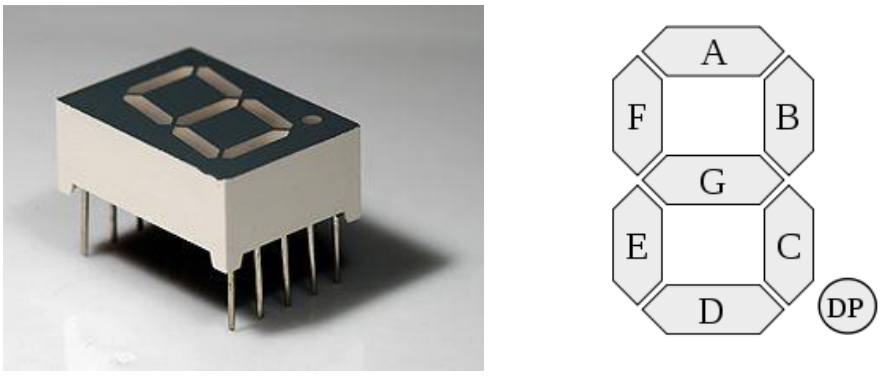

Figura 1. (a) Foto de un display de 7 segmentos. (b) Dibujo un display con los segmentos de A a G.

Así, la salida del decodificador debe ser capaz de mostrar los siguientes dígitos decimales (Figura 2).

Figura 2. Configuración de los segmentos del display para la representación de números BCD.

Por conveniencia, los segmentos *A...G* de la Figura 1 se renombran como *led(0)... led(6)*. Como tenemos 10 configuraciones distintas, se necesitan 4 entradas, pero de ellas no necesitamos todos los valores (16). Los 6 valores restantes serán términos sin importancia (*don't care*) que se emplean para simplificar las ecuaciones lógicas.

El decodificador de 7 segmentos responde a la tabla de verdad que se muestra en la Tabla 1. En nuestro caso, el display de 7 segmentos de la placa de desarrollo a usar tiene lógica negativa. Esto supone que cada led del display se enciende cuando el valor de su segmento es 0, y permanece apagado cuando vale 1. Esta consideración se tiene en cuenta en la tabla 1. Por ejemplo, si se desea representar un 0 en el display como el de la Figura 2 (fila 0 de la Tabla 1), deben encenderse todos los led del display excepto el segmento central (es decir, todos los valores son 0 excepto *led(6)* que vale 1).

bcd(3)	bcd(2)	bcd(1)	bcd(0)	led(6) G	led(5) F	led(4) E	led(3) D	led(2) C	led(1) B	led(0) A
0	0	0	0	1	0	0	0	0	0	0
0	0	0	1	1	1	1	1	0	0	1
0	0	1	0	0	1	0	0	1	0	0
0	0	1	1	0	1	1	0	0	0	0
0	1	0	0	0	0	1	1	0	0	1
0	1	0	1	0	0	1	0	0	1	0
0	1	1	0	0	0	0	0	0	1	0
0	1	1	1	1	1	1	1	0	0	0
1	0	0	0	0	0	0	0	0	0	0
1	0	0	1	0	0	1	0	0	0	0
1	0	1	0	X	X	X	X	X	X	X
1	0	1	1	X	X	X	X	X	X	X
1	1	0	0	X	X	X	X	X	X	X
1	1	0	1	X	X	X	X	X	X	X
1	1	1	0	X	X	X	X	X	X	X
1	1	1	1	X	X	X	X	X	X	X

Tabla 1. Tabla de verdad del display de 7 segmentos

La simplificación de las 7 funciones (una para cada segmento del display) mediante los mapas de Karnaugh darán lugar a las ecuaciones lógicas que se deben describir en VHDL para la creación del circuito lógico en la FPGA. Las ecuaciones simplificadas son:

$$led(6) = \overline{bcd(3)} \cdot \overline{bcd(2)} \cdot \overline{bcd(1)} + bcd(2) \cdot bcd(1) \cdot bcd(0)$$
$$led(5) = bcd(1) \cdot bcd(0) + \overline{bcd(3)} \cdot \overline{bcd(2)} \cdot bcd(0) + \overline{bcd(3)} \cdot \overline{bcd(2)} \cdot bcd(1)$$
$$led(4) = bcd(0) + \overline{bcd(3)} \cdot bcd(2) \cdot \overline{bcd(1)}$$
$$led(3) = \overline{bcd(3)} \cdot \overline{bcd(2)} \cdot bcd(1) \cdot \overline{bcd(0)} + bcd(2) \cdot \overline{bcd(1)} \cdot \overline{bcd(0)} + bcd(2) \cdot bcd(1) \cdot bcd(0)$$
$$led(2) = \overline{bcd(2)} \cdot bcd(1) \cdot \overline{bcd(0)}$$
$$led(1) = bcd(2) \cdot \overline{bcd(1)} \cdot bcd(0) + bcd(2) \cdot bcd(1) \cdot \overline{bcd(0)}$$
$$led(0) = \overline{bcd(3)} \cdot \overline{bcd(2)} \cdot \overline{bcd(1)} \cdot bcd(0) + bcd(2) \cdot \overline{bcd(1)} \cdot \overline{bcd(0)}$$

Los objetivos a alcanzar en esta práctica son:

- Usar de buses y señales internas en la descripción del circuito
- Cambiar la representación numérica en el cronograma de simulación

- Ahondar en la estructura interna de los CLB de las FPGA
- Niveles de abstracción en la descripción VHDL de un circuito
- Implementación en la tarjeta de desarrollo del decodificador de 7 segmentos

2.- Creación del proyecto

Para la creación del proyecto de Vivado de esta actividad, deben seguirse los pasos descritos en el Apartado 3 de la Actividad 0. En este caso, el nombre del proyecto es *Actividad4* (todo junto) y el resto de parámetros se deja igual.

3.- Creación del código VHDL del decodificador

Para la creación del proyecto de Vivado de esta actividad, deben seguirse los pasos descritos en el Apartado 3 de la Actividad 1. En este caso, el nombre del fichero VHDL se puede poner *dec7seg* y, además, debe introducirse los datos de los puertos en el asistente acorde a la Figura 3.

En el caso del decodificador de 7 segmentos, los 4 puertos de entrada están relacionados, ya que conforman un número de 4 bits. Lo mismo sucede con las 7 salidas del circuito, que van a conformar un número del 0 al 9 en el display. Este tipo de entradas y salidas se suele agrupar conformando un *bus*. La Figura 3 muestra que se van a crear un circuito con un puerto de entrada de 4 bits y un puerto de salida de 7 bits. Para hacer esto, se marca en la declaración de los puertos la casilla de bus, y a continuación se introduce el tamaño del mismo. MSB es el acrónimo de bit más significativo en inglés (*most significant bit*) y LSB, bit menos significativo (*least significant bit*). Normalmente, la columna LSB se deja a 0, definiendo el tamaño del bus con el valor de MSB, de tal manera que hay que considerar que el tamaño final del bus suele ser igual a MSB+1.

La plantilla VHDL creada se muestra en la Figura 4. Se puede apreciar en la entidad la sintaxis para definir estos puertos compuestos mediante STD_LOGIC_VECTOR. El tamaño en bits se especifica entre paréntesis.

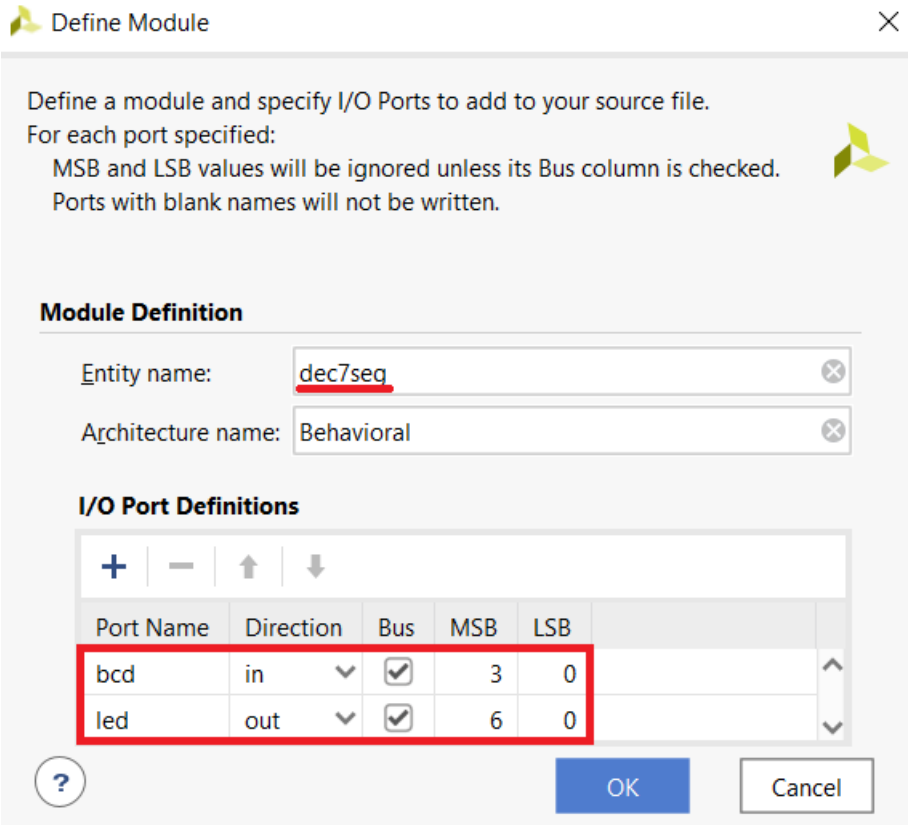

Figura 3. Captura de pantalla de la ventana Define Module con los buses definimos resaltados en rojo.

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity dec7seg is
5      Port ( bcd : in STD_LOGIC_VECTOR (3 downto 0);
6             led : out STD_LOGIC_VECTOR (6 downto 0));
7  end dec7seg;
8
9  architecture Behavioral of dec7seg is
10
11 begin
12
13 end Behavioral;
```

Figura 4. Captura de pantalla de la plantilla VHDL generada con los buses de la entidad resaltados en rojo.

El siguiente paso consiste en describir el decodificador de 7 segmentos cuyo comportamiento se rige por las ecuaciones del Apartado 1. El circuito asociado a dichas ecuaciones se muestra en la Figura 5.

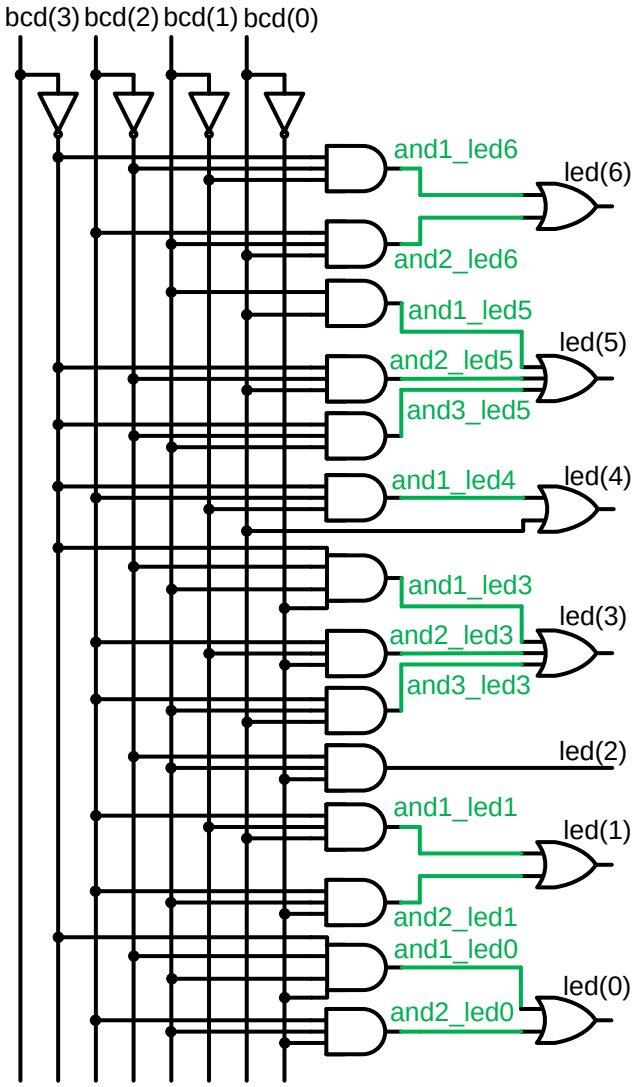

Figura 4. Circuito del decodificador de 7 segmentos.

En esta ocasión, para que el código VHDL sea lo más legible posible, se hará la descripción del circuito de la Figura 4 por partes, haciendo uso de señales internas. Estas señales internas no son más que cables que conectan las salidas de unas puertas lógicas con las entradas de otras puertas lógicas. En la Figura 4 aparecen de color verde.

La arquitectura del decodificador de 7 segmentos se muestra en la Figura 5. Se puede observar que:

- En la zona declarativa de la arquitectura se han definido las señales internas que se van a emplear, donde las líneas de código han sido tabuladas para mayor legibilidad.
- En la zona funcional se ha descrito el circuito. Primero se ha realizado las asignaciones concurrentes que describen los productos lógicos y, a continuación, las asignaciones a los puertos de salida del circuito. Es importante observar que, como las señales internas conectan salidas

y entradas de puertas lógicas, aparecen tanto a la izquierda como a la derecha en las asignaciones.

```
 9 architecture Behavioral of dec7seg is
10     signal and1_led6, and2_led6              : std_logic;
11     signal and1_led5, and2_led5, and3_led5 : std_logic;
12     signal and1_led4                         : std_logic;
13     signal and1_led3, and2_led3, and3_led3 : std_logic;
14     signal and1_led1, and2_led1              : std_logic;
15     signal and1_led0, and2_led0              : std_logic;
16
17 begin
18
19     and1_led6 <= not bcd(3) and not bcd(2) and not bcd(1);
20     and2_led6 <= bcd(2) and bcd(1) and bcd(0);
21     and1_led5 <= bcd(1) and bcd(0);
22     and2_led5 <= not bcd(3) and not bcd(2) and bcd(0);
23     and3_led5 <= not(bcd(2)) and bcd(1);
24     and1_led4 <= bcd(2) and not bcd(1);
25     and1_led3 <= not bcd(3) and not bcd(2) and bcd(0) and not bcd(1);
26     and2_led3 <= bcd(2) and not bcd(1) and not bcd(0);
27     and3_led3 <= bcd(2) and bcd(1) and bcd(0);
28     and1_led1 <= bcd(2) and not bcd(1) and bcd(0);
29     and2_led1 <= bcd(2) and bcd(1) and not(bcd(0));
30     and1_led0 <= not bcd(3) and not bcd(2) and bcd(0) and not bcd(1);
31     and2_led0 <= bcd(2) and not bcd(1) and not bcd(0);
32
33     led(6) <= and1_led6 or and2_led6;
34     led(5) <= and1_led5 or and2_led5 or and3_led5;
35     led(4) <= bcd(0) or and1_led4;
36     led(3) <= and1_led3 or and2_led3 or and3_led3;
37     led(2) <= not bcd(2) and bcd(1) and not bcd(0);
38     led(1) <= and1_led1 or and2_led1;
39     led(0) <= and1_led0 or and2_led0;
40 end Behavioral;
```

Figura 5. Captura de pantalla de la arquitectura de *dec7seg.vhd*.

Es importante recordar que el orden en el que figuran las asignaciones en la arquitectura es irrelevante. Se pueden poner en cualquier orden.

4.- Simulación del decodificador

La simulación del decodificador requiere la creación del *test bench* correspondiente. Para la creación *test bench*, deben seguirse los pasos descritos en el Apartado 6 de la Actividad 1. En este caso, el nombre del *test*

bench se puede poner *dec7seg_tb*. La arquitectura se muestra en la Figura 6.

```
 8  architecture Behavioral of dec7seg_tb is
 9
10      component dec7seg
11          port (bcd : in std_logic_vector (3 downto 0);
12                led : out std_logic_vector (6 downto 0));
13      end component;
14
15      signal bcd : std_logic_vector (3 downto 0) := (others => '0');
16  --    signal bcd : std_logic_vector (3 downto 0) := X"0";
17  --    signal bcd : std_logic_vector (3 downto 0) := "0000";
18      signal led : std_logic_vector (6 downto 0);
19
20      constant Tb_semiPeriod : time := 10 ns; -- Put right period here
21      signal TbSimEnded : std_logic := '0';
22
23  begin
24
25      dut : dec7seg
26      port map (bcd => bcd,
27                led => led);
28
29      -- Stimuli generation
30      bcd(0) <= not bcd(0) after Tb_semiPeriod   when TbSimEnded /= '1' else '0';
31      bcd(1) <= not bcd(1) after Tb_semiPeriod*2 when TbSimEnded /= '1' else '0';
32      bcd(2) <= not bcd(2) after Tb_semiPeriod*4 when TbSimEnded /= '1' else '0';
33      bcd(3) <= not bcd(3) after Tb_semiPeriod*8 when TbSimEnded /= '1' else '0';
34
35      -- End of simulation: multiply Tb_semiPeriod by 2^nº variables
36      TbSimEnded <= '1' after 16 * Tb_semiPeriod; -- 2^4=16
37
38  end Behavioral;
```

Figura 6. Captura de pantalla de la arquitectura de *dec7seg_tb.vhd*.

Respecto a los *test bench* de las actividades anteriores, hay que considerar los siguientes aspectos:

- Para inicializar la señal interna tipo std_logic_vector *bcd* existen 3 maneras distintas (líneas 15, 16 y 17). La primera es la más usada, ya que inicializa todos los bits del bus a cero, independientemente del tamaño. Las otras dos opciones inicializan el bus con un valor en hexadecimal y en binario, respectivamente.
- Para variar los estímulos de entrada y cubrir todas las combinaciones, *bcd(0)* se considera la entrada menos significativa; y *bcd(3)*, las más significativa (la que varía más lentamente).

El hecho de agrupar en el diseño los puertos de entrada y de salida sirve para comprobar el correcto funcionamiento del circuito mediante el cronograma de simulación de una manera más rápida y sencilla. Para ello, se modifica la tabla de verdad de la Tabla 1 para representar la entrada en

formato decimal sin signo y la salida en formato hexadecimal, teniendo en cuenta que cada cuatro dígitos binarios se corresponden con un dígito hexadecimal. El resultado de esta conversión se muestra en la Tabla 2.

bcd decimal	bcd binario	led(6) G	led(5) F	led(4) E	led(3) D	led(2) C	led(1) B	led(0) A	led hexa
0	0000	1	0	0	0	0	0	0	40
1	0001	1	1	1	1	0	0	1	79
2	0010	0	1	0	0	1	0	0	24
3	0011	0	1	1	0	0	0	0	30
4	0100	0	0	1	1	0	0	1	19
5	0101	0	0	1	0	0	1	0	12
6	0110	0	0	0	0	0	1	0	02
7	0111	1	1	1	1	0	0	0	78
8	1000	0	0	0	0	0	0	0	00
9	1001	0	0	1	0	0	0	0	10
10	1010	X	X	X	X	X	X	X	XX
11	1011	X	X	X	X	X	X	X	XX
12	1100	X	X	X	X	X	X	X	XX
13	1101	X	X	X	X	X	X	X	XX
14	1110	X	X	X	X	X	X	X	XX
15	1111	X	X	X	X	X	X	X	XX

Tabla 2. Tabla de verdad del display de 7 segmentos con las entradas en formato decimal sin signo y las salidas en formato hexadecimal.

El cronograma de simulación correspondiente al *test bench* del decodificador se muestra en la Figura 7. Al trabajar con *std_logic_vector*, las entradas y las salidas aparecen agrupadas en el cronograma. Para ver el valor de cada bit por separado, debe hacerse clic en el icono en rojo.

Cada uno de los 2 buses (*bcd* y *led*), se pueden poner en distintas representaciones numéricas en el cronograma. En este caso, interesa poner la entrada como entero sin signo y la salida en hexadecimal. Para realizar estas operaciones, hay que hacer clic con el botón derecho del ratón sobre el nombre del bus cuya representación se desee cambiar, y, en el menú emergente, seleccionar *Radix*, y *unsigned decimal* para el bus *bcd* y *hexadecimal* para el bus *led*. Con esto, se puede contrastar mucho más fácil los valores obtenidos con los de la Tabla 2.

Los valores de salida obtenidos para las entradas *bcd* desde 10 (*1010*) hasta 15 (*1111*), son irrelevantes, ya las entradas están restringidas a números del 0 al 9. Los valores que se obtienen para estas entradas en el cronograma según dependen de si se escogen o no los términos *don't care* en los mapas de Karnaugh correspondientes a las funciones de la Tabla 1 para obtener las funciones simplificadas.

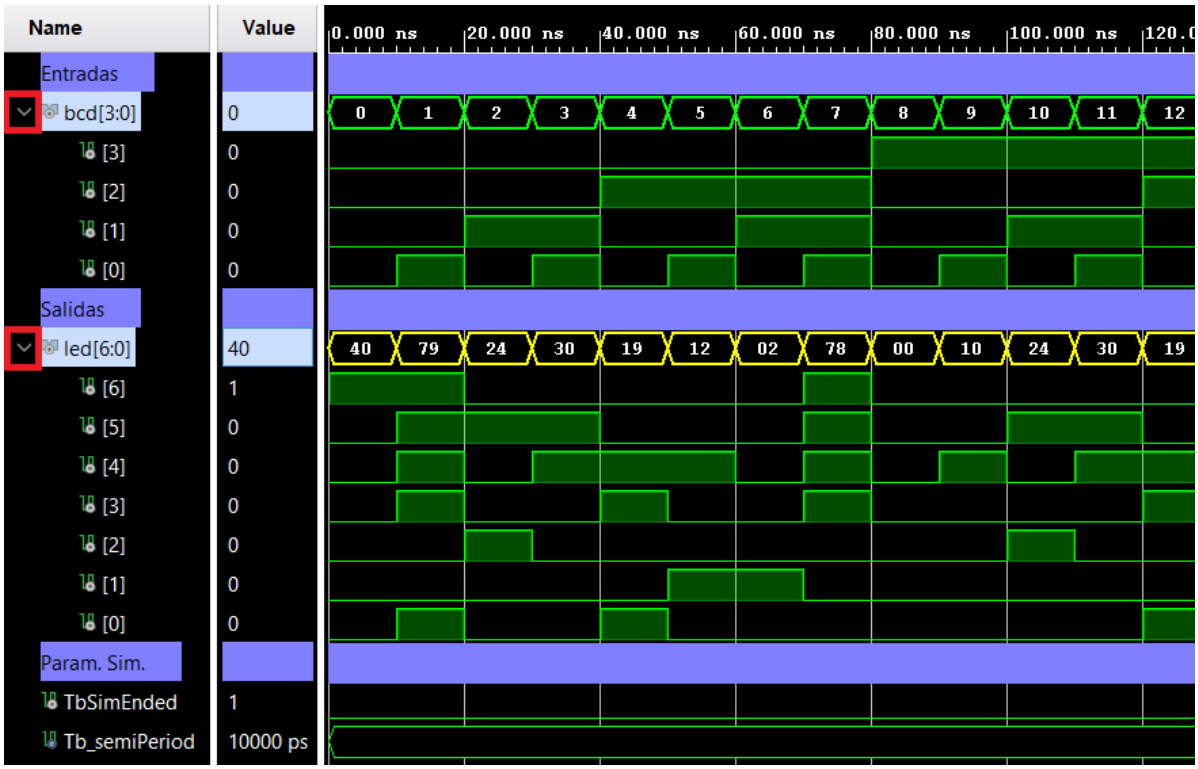

Figura 7. Captura de pantalla del cronograma de simulación del decodificador de 7 segmentos.

5.- Asignación de pines de la FPGA

Para asignar qué pines de la FPGA se van a conectar a los puertos de entrada y salida del decodificador de 7 segmentos hay que seguir el procedimiento especificado del Apartado 3 y siguientes de la Actividad 3. En este caso, las entradas se van a conectar a los pines de la FPGA que están conectados a los 4 interruptores más a la derecha de la tarjeta, y las salidas, a los pines de la FPGA conectados a los segmentos de los displays de 7 segmentos. Dichos pines se muestran en la Figura 8 [1].

En el esquema de la Figura 8 se aprecia que, con el objetivo de ahorrar pines en la FPGA, todos los segmentos iguales de los displays están conectados al mismo pin de la FPGA (por ejemplo, todos los segmentos *CA* están conectados al pin T10). Para seleccionar el display activo, se hace uso de las conexiones *AN7*... *AN0*. Por ejemplo, si se quiere mostrar un número sólo en el display de la derecha del todo, sólo *AN0* debe ser 0 lógico y los demás, a 1 lógico. En el ejemplo que nos ocupa, no seleccionaremos ningún display en concreto, así que la salida se mostrará en todos los displays simultáneamente. De esta manera, en lugar de emplear 64 pines para controlar los 8 displays, se emplean únicamente 16. La contrapartida es que

no es posible representar 8 valores distintos simultáneamente cada uno en un display.

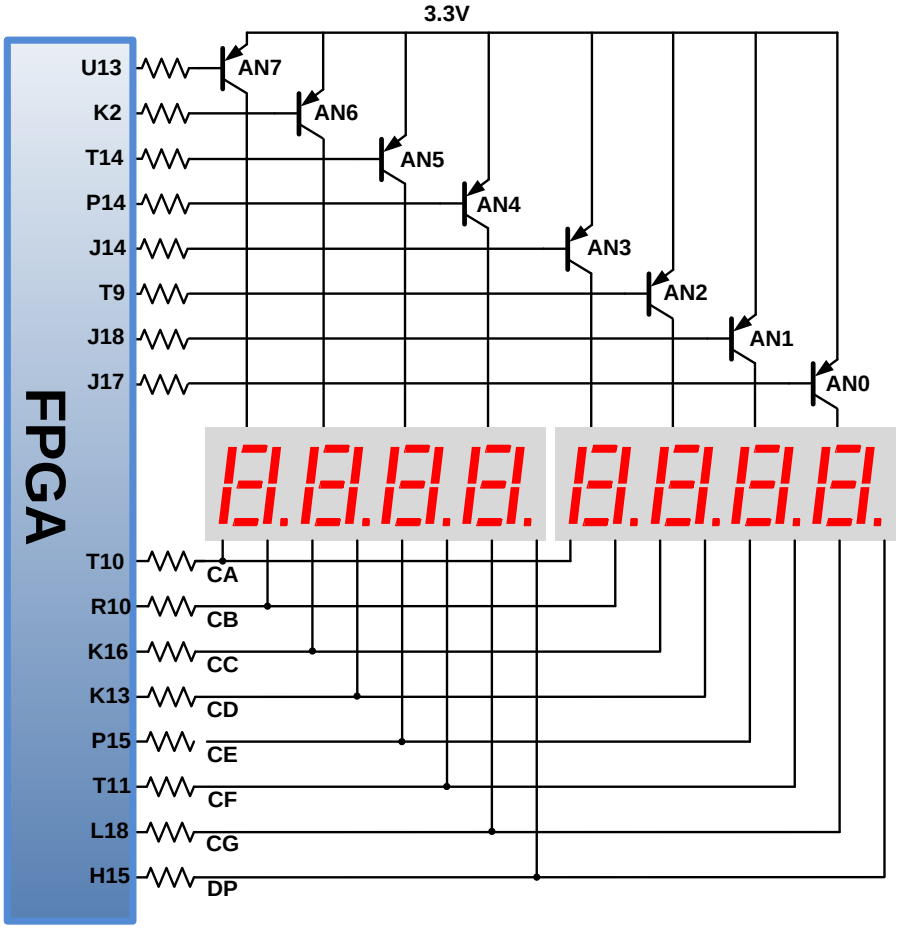

Figura 8. Esquema de conexiones de los displays de 7 segmentos con los pines de la FPGA de la tarjeta Nexys A7.

Teniendo en cuenta que sólo se va a representar un dato en todos los displays, se importa la plantilla maestra de asignación de pines y se realizan las modificaciones pertinentes para este diseño (Figura 9). La plantilla maestra la provee el fabricante de la tarjeta [2].

```
11  ##Switches
12  set_property -dict { PACKAGE_PIN J15   IOSTANDARD LVCMOS33 } [get_ports { bcd[0] }];
13  set_property -dict { PACKAGE_PIN L16   IOSTANDARD LVCMOS33 } [get_ports { bcd[1] }];
14  set_property -dict { PACKAGE_PIN M13   IOSTANDARD LVCMOS33 } [get_ports { bcd[2] }];
15  set_property -dict { PACKAGE_PIN R15   IOSTANDARD LVCMOS33 } [get_ports { bcd[3] }];
16
55  ##7 segment display
56  set_property -dict { PACKAGE_PIN T10   IOSTANDARD LVCMOS33 } [get_ports { led[0] }];
57  set_property -dict { PACKAGE_PIN R10   IOSTANDARD LVCMOS33 } [get_ports { led[1] }];
58  set_property -dict { PACKAGE_PIN K16   IOSTANDARD LVCMOS33 } [get_ports { led[2] }];
59  set_property -dict { PACKAGE_PIN K13   IOSTANDARD LVCMOS33 } [get_ports { led[3] }];
60  set_property -dict { PACKAGE_PIN P15   IOSTANDARD LVCMOS33 } [get_ports { led[4] }];
61  set_property -dict { PACKAGE_PIN T11   IOSTANDARD LVCMOS33 } [get_ports { led[5] }];
62  set_property -dict { PACKAGE_PIN L18   IOSTANDARD LVCMOS33 } [get_ports { led[6] }];
```

Figura 9. Captura de pantalla las líneas del archivo de restricciones correspondientes a los puertos del circuito *dec7seg.vhd*.

6.- Configuración del decodificador en la FPGA

El último paso consiste en configurar la tarjeta con el diseño del decodificador y haciendo uso de fichero de restricciones de usuario con los pines asignados a los puertos. Para realizar este paso hay que proceder de la misma manera que lo explicado en el Apartado 5 y el Apartado 6 de la Actividad 3.

La Figura 10 muestra muestra la tarjeta de desarrollo con el decodificador implementado en la FPGA Artix-7. En este caso, a través de los interruptores se ingresa el valor 0111 y se observa el número 7 en los displays.

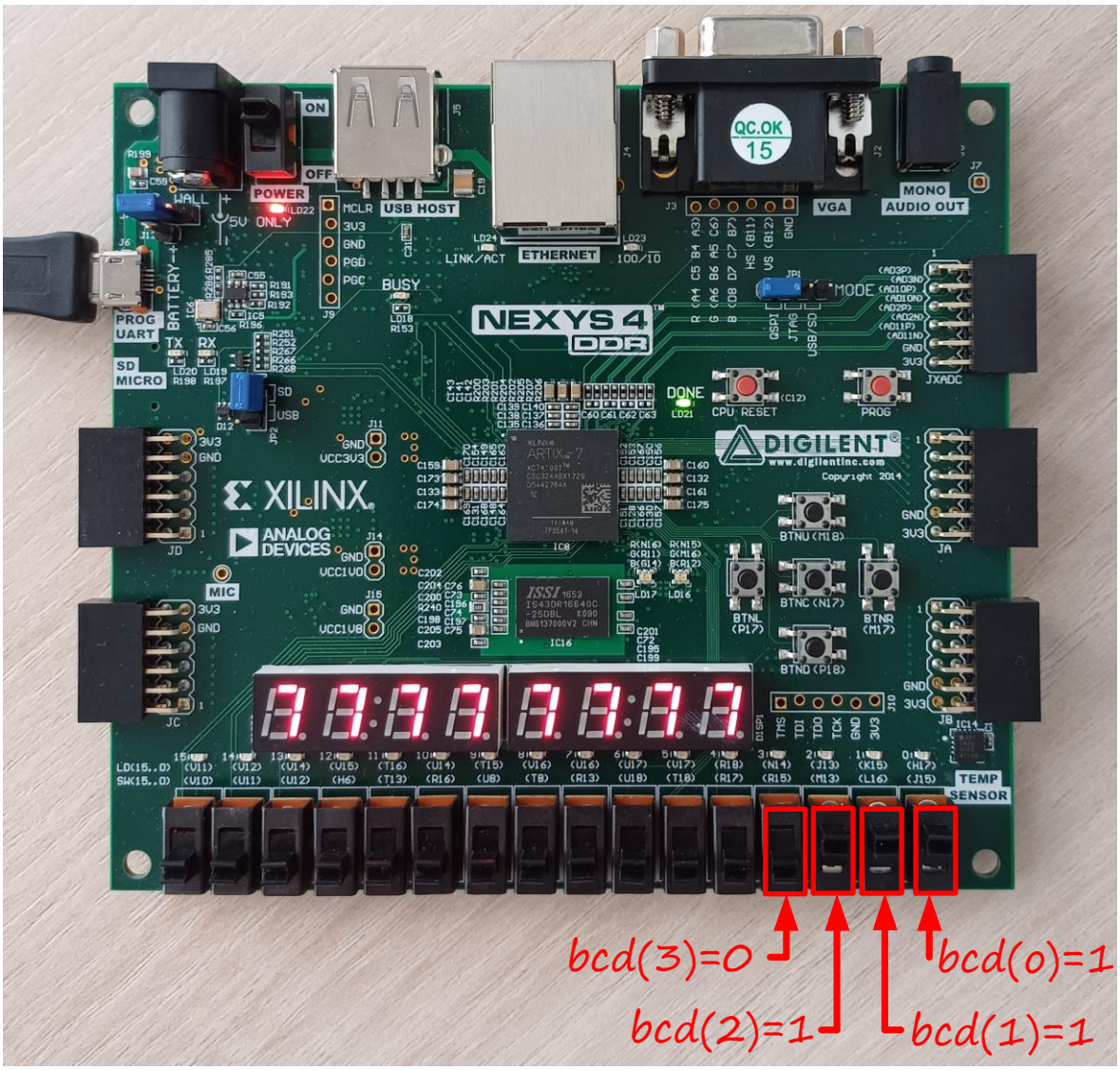

Figura 10. Foto de la tarjeta de desarrollo con el decodificador implementado en la FPGA.

7.- Niveles de abstracción en VHDL

Uno de los puntos fuertes del lenguaje VHDL es su versatilidad en el sentido que un mismo circuito se puede describir con más o menos detalle, que ya se encarga el sintetizador y las herramientas de implementación en inferir el circuito para una FPGA determinada. Generalmente, se considera aceptado el hecho de que VHDL comprende tres niveles de abstracción según el nivel de detalle de la descripción del circuito:

- **Nivel algorítmico, funcional o comportamental:** se describe el circuito a alto nivel, con una sintaxis que puede recordar a un lenguaje software, con sentencias tipo *when…else*, *with…select*, *if…then*, etcétera.

  Esta manera de describir el circuito es la más compacta de todas (requiere de menos líneas de código). No obstante, hay que ser cuidadoso en describir hardware en este nivel de abstracción para no caer una mala descripción viciada por una programación orientada a software (muchos *if* anidados y cosas así), dando lugar a circuitos no óptimos.

  Su descripción análoga a la programación hardware hace que sea una descripción hardware con muy poca confidencialidad (es muy fácil realizar ingeniería inversa).

- **Nivel RTL (*Register-Transfer Level*)**: se describe el circuito con registro y ecuaciones lógicas. Los diseños que se han implementado en las actividades, si bien son puramente combinaciones, se enmarcan en este nivel.

  Su nivel de confidencialidad es mayor que el nivel algorítmico, y su nivel de detalle mayor, por lo que la descripción VHDL de los circuitos requiere mayor número de líneas de código.

- **Nivel estructural**: en este nivel se describe el circuito como una *netlist* empleando explícitamente los componentes intrínsecos de la FPGA en la que se va a realizar el circuito. Es el nivel que más detalle incluye en la especificación de los diseños y es dependiente de la tecnología (depende del fabricante de FPGA, y de la familia dentro de las FPGA del propio fabricante, ya que sus componentes internos son distintos).

  Es el nivel cuya codificación VHDL ocupa más (diseño complejos conllevan cientos de miles de líneas de código), y el más confidencial de las tres alternativas.

La Figura 11 muestra un esquema de los niveles de abstracción en el diseño usando VHDL. Hay que tener en cuenta que, en cualquier momento en la fase de diseño, se puede cambiar de nivel de abstracción. E incluso en

un mismo circuito, podemos encontrar partes o subcircuitos descritos usando diferentes niveles de abstracción.

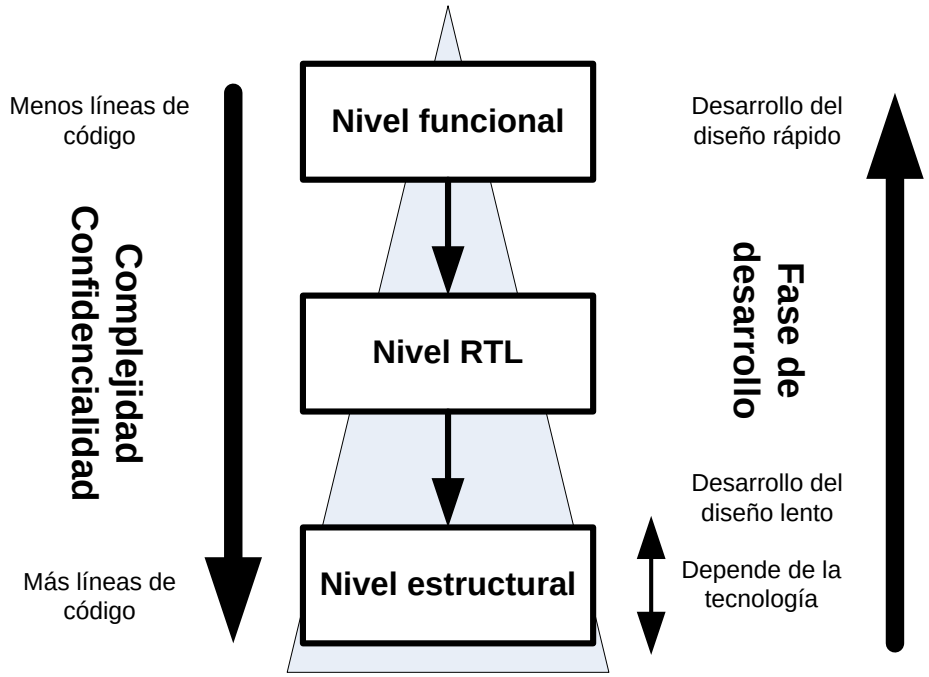

Figura 11. Niveles de abstracción en el diseño usando VHDL.

8.- Descripción funcional del decodificador de 7 segmentos

En el proyecto de la Actividad4 (el actual), se debe crear una *New Source* denominada *dec7seg_funcional.vhd*. La entidad de este nuevo circuito no cambia (sólo la descripción en la arquitectura), así que se debe seguir los pasos del apartado 2 para generar la plantilla VHDL.

A continuación, se describe la arquitectura en un nivel algorítmico, funcional o comportamental. Esta porción del código se muestra en la Figura 12.

Efectivamente, si se compara con la descripción a nivel RTL (Figura 5), se puede apreciar que el código es más compacto (ocupa menos líneas de código); y es menos confidencial (se puede extraer fácilmente el cometido del circuito analizando el código).

```
10 architecture Behavioral of dec7seg_funcional is
11
12 begin
13 -- segment encoinputg
14 --       0
15 --      ---
16 --   5 |   | 1
17 --      ---   <- 6
18 --   4 |   | 2
19 --      ---
20 --       3
21
22      with bcd select
23     led<= "1000000" when "0000",   --0
24           "1111001" when "0001",   --1
25           "0100100" when "0010",   --2
26           "0110000" when "0011",   --3
27           "0011001" when "0100",   --4
28           "0010010" when "0101",   --5
29           "0000010" when "0110",   --6
30           "1111000" when "0111",   --7
31           "0000000" when "1000",   --8
32           "0010000" when "1001",   --9
33           "0111111" when others;   ---
34
35 end Behavioral;
```

Figura 11. Captura de pantalla de la arquitectura de *dec7seg_funcional.vhd* (nivel de abstracción funcional).

El hecho de que la entidad del circuito no cambie, hace que el *test bench* creado en el apartado 4 se pueda adaptar fácilmente si se requiere simular este circuito (únicamente hay que sustituir las referencias del nombre del circuito por el actual (líneas 10 y 25 en la Figura 6). También es posible crear un nuevo *test bench*, con lo que se tendrían en el proyecto dos, uno para cada descripción del decodificador.

Para activar el nuevo circuito como el circuito activo en el proyecto, se usa la opción *Set as Top*, según lo comentado en el apartado 6 de la Actividad 3. Haciendo esto, la jerarquía del circuito debe quedar como se muestra en la Figura 12.

Respecto a la asignación de pines, el fichero que se ha usado es válido para el circuito activo actual, ya que no ha cambiado la interfaz en la entidad y tanto las entradas como las salidas tienen el mismo cometido (no se va a modificar los periféricos de la tarjeta a emplear).

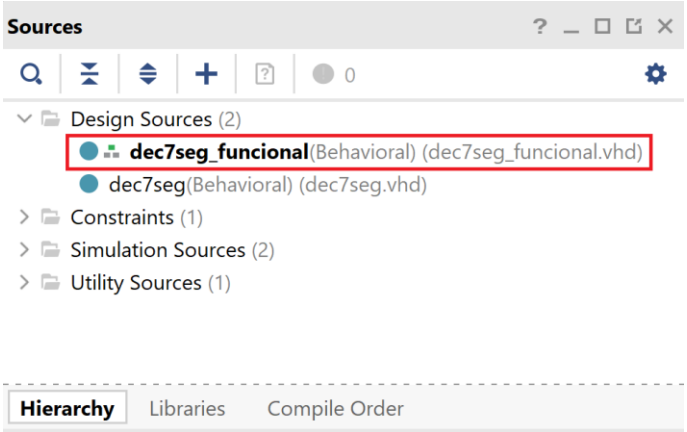

Figura 12. Captura de pantalla de la pestaña de jerarquía con el nuevo circuito activo como *top level* resaltado en rojo.

Para configurar la tarjeta sólo queda generar el archivo .BIT. El entorno de desarrollo, a través de los procesos previos de síntesis e implementación, extraerá la información de alto nivel del código VHDL funcional para obtener la información para la configuración de los CLB de la FPGA de manera automática y transparente por parte del ingeniero electrónico. El procedimiento para generar el .BIT y configurar la tarjeta se detalla en el Apartado 4 y en el Apartado 5 de la Actividad 3.

El resultado debe ser igual que el que se mostró en la Figura 10, cuando se configura la tarjeta con el archivo .BIT generado.

Nota: en realidad el circuito generado es ligeramente distinto, ya que para las combinaciones de entrada de *1010* a *1111*, la salida descrita en el código hace dibujar una raya central en los displays).

9.- Descripción estructural del decodificador de 7 segmentos

Básicamente hay que seguir los pasos del apartado 8, pero definiendo el nuevo código como *dec7seg_estructural.vhd*. Es importante seleccionar este circuito como top level en esta ocasión (Figura 13).

La Figura 14 muestra las librerías y la entidad de este nuevo código VHDL. Para descripciones en el nivel de abstracción estructural, debe estar siempre la librería UNISIM, en la que están definidos todos los componentes internos de la FPGA. Por otra parte, se aprecia que la entidad es la misma que en las dos descripciones anteriores (RTL y funcional).

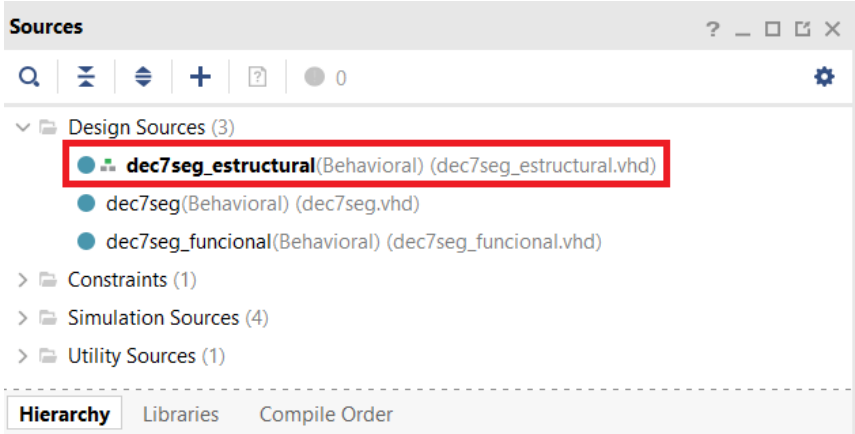

Figura 13. Captura de pantalla de la pestaña de jerarquía con el nuevo circuito activo como *top level* resaltado en rojo.

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  -- Uncomment the following library declaration if instantiating
5  -- any Xilinx leaf cells in this code.
6  library UNISIM;
7  use UNISIM.VComponents.all;
8
9  entity dec7seg_estructural is
10     Port ( bcd : in STD_LOGIC_VECTOR (3 downto 0);
11            led : out STD_LOGIC_VECTOR (6 downto 0));
12 end dec7seg_estructural;
```

Figura 14. Captura de pantalla de las librerías y la entidad del código VHDL de la descripción estructural del decodificador de 7 segmentos.

La arquitectura de la descripción del circuito a nivel estructural hace uso del componente LUT4, que se adapta al componente LUT6 que forma parte los CLB de las FPGA de la familia 7 de Xilinx. La descripción y el empleo de éste y otros componentes está documentada por Xilinx en documento *Vivado Design Suite 7 Series FPGA and Zynq 7000 SoC Libraries Guide* (UG953) [3].

El código VHDL de la arquitectura estructural del decodificador de 7 segmentos se muestra en la Figuras 15 y 16. Sin entrar en detalles, el componente LUT4 que aparece viene a ser una memoria de 16 bits a la que se accede con 4 entradas. En cada LUT4 se mapea con el PORT MAP (igual que en los *test bench*) los puertos de entrada (bus *bcd*) y el puerto de salida (puerto *led*) con los puertos internos del componente. El contenido de la memoria de 16 bits se configura a través del *generic map*, con el valor en hexadecimal de la tabla de verdad de cada función lógica (Tabla 1).

```
14 architecture Behavioral of dec7seg_estructural is
15
16 begin
17
18    -- LUT4: 4-input Look-Up Table with general output
19    --       Artix-7
20    -- Xilinx HDL Language Template, version 2023.1
21
22    LUT4_inst0 : LUT4
23    generic map (
24       INIT => X"0012")
25    port map (
26       O => led(0),    -- LUT general output
27       I0 => bcd(0), -- LUT input
28       I1 => bcd(1), -- LUT input
29       I2 => bcd(2), -- LUT input
30       I3 => bcd(3)  -- LUT input
31    );
32    LUT4_inst1 : LUT4
33    generic map (
34       INIT => X"0060")
35    port map (
36       O => led(1),    -- LUT general output
37       I0 => bcd(0), -- LUT input
38       I1 => bcd(1), -- LUT input
39       I2 => bcd(2), -- LUT input
40       I3 => bcd(3)  -- LUT input
41    );
42    LUT4_inst2 : LUT4
43    generic map (
44       INIT => X"0004")
45    port map (
46       O => led(2),    -- LUT general output
47       I0 => bcd(0), -- LUT input
48       I1 => bcd(1), -- LUT input
49       I2 => bcd(2), -- LUT input
50       I3 => bcd(3)  -- LUT input
51    );
52    LUT4_inst3 : LUT4
53    generic map (
54       INIT => X"0092")
55    port map (
56       O => led(3),    -- LUT general output
57       I0 => bcd(0), -- LUT input
58       I1 => bcd(1), -- LUT input
59       I2 => bcd(2), -- LUT input
60       I3 => bcd(3)  -- LUT input
61    );
```

Figura 15. Captura de pantalla de la arquitectura de *dec7seg_estructural.vhd*.

```
62      LUT4_inst4 : LUT4
63      generic map (
64         INIT => X"02BA")
65      port map (
66         O => led(4),    -- LUT general output
67         I0 => bcd(0), -- LUT input
68         I1 => bcd(1), -- LUT input
69         I2 => bcd(2), -- LUT input
70         I3 => bcd(3)  -- LUT input
71      );
72      LUT4_inst5 : LUT4
73      generic map (
74         INIT => X"008E")
75      port map (
76         O => led(5),    -- LUT general output
77         I0 => bcd(0), -- LUT input
78         I1 => bcd(1), -- LUT input
79         I2 => bcd(2), -- LUT input
80         I3 => bcd(3)  -- LUT input
81      );
82      LUT4_inst6 : LUT4
83      generic map (
84         INIT => X"0083")
85      port map (
86         O => led(6),    -- LUT general output
87         I0 => bcd(0), -- LUT input
88         I1 => bcd(1), -- LUT input
89         I2 => bcd(2), -- LUT input
90         I3 => bcd(3)  -- LUT input
91      );
92
93   end Behavioral;
```

Figura 16. Captura de pantalla de la arquitectura de *dec7seg_estructural.vhd* (continuación).

Es importante realizar el mapeo del componente LUT4 de manera correcta, teniendo en cuenta que la entrada *I3* es la más significativa e *I0* la menos significativa [3].

Puede comprobarse, analizando el código, que, por una parte, se trata la descripción que más líneas de código necesita de las tres alternativas, y, por otra parte, que resulta muy difícil averiguar qué circuito realiza el código sólo analizándolo (alta confidencialidad).

Por último, este circuito puede simularse y configurarse en la FPGA de la tarjeta procediendo de manera análoga a lo que se detalla en el Apartado 8.

Referencias

[1] https://digilent.com/reference/programmable-logic/nexys-a7/reference-manual
[2] https://github.com/Digilent/digilent-xdc/blob/master/Nexys-A7-100T-Master.xdc
[3] https://docs.xilinx.com/r/en-US/ug953-vivado-7series-libraries